Assignment 1: Data Privacy

Data Protection Methods on the Adult Dataset

1. Introduction

This report covers data protection methods applied to the UCI Adult dataset. The dataset has census info for about 32,000 people and is a standard benchmark for classification - the goal being to predict whether someone earns over $50K/year. It has a mix of numerical and categorical attributes like age, education, occupation, etc.

I tried three masking methods: generalization (rounding quasi-identifiers), rank swapping, and Laplace noise addition. For each one I tested four parameter settings to see how privacy and utility trade off. Evaluation was done through disclosure risk (distance-based record linkage), generic information loss, and classification accuracy (Random Forest on income).

2. Data Preprocessing

The raw dataset had 32,561 records with 15 columns (14 features + income). Some rows had missing values (marked with "?") in workclass, occupation, and native_country. I dropped those, leaving 30,162 complete records.

Categorical columns were label-encoded using scikit-learn. Income was mapped manually (<=50K to 0, >50K to 1). I used LabelEncoder instead of one-hot to keep the column count low - one-hot would have blown up the number of features and made distance calculations more complicated later.

After encoding, all 14 feature columns were normalized to [0,1] with MinMaxScaler. Without this, the Euclidean distance in the disclosure risk step would be dominated by columns like fnlwgt (which goes up to hundreds of thousands) while something like education_num only ranges from 1 to 16.

3. Masking Methods

3.1 Heuristic Generalization

Generalization here means rounding the normalized values to fewer decimal places. A "depth" of 1 means round to 1 decimal, depth 0 rounds to the nearest integer (so in the [0,1] range everything becomes 0 or 1). I applied this to age and education_num since those are typical quasi-identifiers.

One thing that became obvious pretty quickly is that this only modifies 2 out of 14 columns. The remaining 12 are completely untouched, so an attacker can still use them for linkage. Only the most aggressive setting (depth 0) actually reduced risk in a meaningful way.

Parameters tested:

- Depth {age: 0, education_num: 0}: most aggressive, collapses to 2 unique values each
- Depth {age: 1, education_num: 1}: moderate, ~11 unique values per column
- Depth {age: 1, education_num: 2}: mixed, 11 for age and 16 for education

- Depth {age: 2, education_num: 2}: least aggressive, 72 unique age values and 16 education

3.2 Rank Swapping

Rank swapping works column by column. Sort the values by rank, then swap each value with another one whose rank is within p% of the total record count. So with p=10% and 30k records, a record at rank 5000 could be swapped with anything between rank 5001 and 8000.

The nice thing about this method is that it preserves marginal distributions exactly - the set of values in each column stays the same, they just get rearranged. Correlations between columns do get disrupted though since each column is swapped independently. Unlike generalization, this hits all 14 columns.

Parameters tested (p = max rank distance as % of N):

- p = 2%: mean absolute difference = 0.0097
- p = 5%: mean absolute difference = 0.0227
- p = 10%: mean absolute difference = 0.0413 (selected)
- p = 20%: mean absolute difference = 0.0741

3.3 Noise Addition (Laplace)

Laplace noise added to every value, then clipped to [0,1]. I went with Laplace over Gaussian because of the heavier tails - outlier records (which tend to be the easiest to re-identify) are more likely to get a large enough perturbation to actually protect them. The scale parameter controls how much noise gets added.

After adding noise I clip values to stay in [0,1]. This does introduce a small bias near the boundaries but most values aren't right at 0 or 1 so it's not a big deal.

Parameters tested (Laplace scale):

- scale = 0.01: very light, mean abs difference = 0.0082
- scale = 0.05: moderate, mean abs difference = 0.0400
- scale = 0.10: moderate-high, mean abs difference = 0.0772 (selected)
- scale = 0.20: heavy, mean abs difference = 0.1406

4. Disclosure Risk Assessment

I measured disclosure risk with distance-based record linkage. For each masked record, find its nearest neighbor in the original data using Euclidean distance across all 14 features. If the nearest neighbor has the same index, that's a successful link - the record could be re-identified. Risk = fraction of correctly linked records.

I used a sample of 3,000 records instead of the full 30k because the full distance matrix would need about 7 GB of memory.

Results for the selected parameters:

- Generalization (depth 0,0): risk = 31.9%
- Rank Swapping (p = 10%): risk = 35.8%
- Noise Addition (scale = 0.1): risk = 33.4%

All three end up in a similar range (32-36%). The original unmasked data has 100% risk by definition since every record matches itself.

5. Generic Information Loss

Information loss was measured using the formula from the assignment: IL(o, m) = (1/|V|) * sum_i sum_v ((o_iv - m_iv)^2 / mean_v). I also computed MSE as a secondary metric.

Results:

Method	IL	MSE
Generalization	1132.60	0.016002
Rank Swapping	3366.75	0.013036
Noise Addition	4169.82	0.014188

IL and MSE tell slightly different stories. Generalization has the highest MSE (0.016) but lowest IL (1132.6) - this is because the IL formula divides by column mean, and since generalization only touches 2 columns, the other 12 contribute zero to the sum. Rank swapping and noise spread distortion across all columns so they get higher IL values but comparable MSE. The MSE values are all pretty small (0.013-0.016), so none of the methods completely wreck the data.

6. Specific Information Loss (Classification)

I trained a Random Forest (100 trees, max depth 15) on the masked data to predict income. Same 70/30 split for all experiments.

Results:

Method	Accuracy	Precision	Recall
Original (baseline)	0.8611	0.7825	0.6119
Generalization	0.8567	0.7807	0.5897
Rank Swapping	0.8225	0.7001	0.5018
Noise Addition	0.7939	0.6418	0.3890

Generalization barely affects accuracy (0.857 vs 0.861), which makes sense - 12 out of 14 features are unchanged. Rank swapping drops to 0.823 which is still reasonable. Noise addition takes the biggest hit, down to 0.794, and the recall especially suffers (0.389 vs 0.612 baseline). So the noisy classifier misses a lot of the >50K income people - it struggles with the minority class.

7. Discussion and Conclusion

Generalization was kind of disappointing. The idea of rounding quasi-identifiers is simple and easy to understand, but only modifying 2 columns out of 14 doesn't do much. Even the most aggressive rounding only got risk to 32%

because an attacker still has the other 12 columns to work with. You'd have to generalize way more columns to get real protection, but then the data would lose too much detail. The accuracy staying high (0.857) is basically a side effect of not doing enough.

Rank swapping worked better than I initially expected. p=10% brought risk down to 36% and accuracy stayed at 0.823. Since it only rearranges existing values, the column-level statistics stay accurate - means, distributions, everything. The classifier can still pick up on the patterns even though individual records are shuffled. I think this makes it the most practical option if you need to release data that people will do statistical analysis on.

Noise addition gives the lowest risk (33%) but also the worst accuracy. The recall drop was pretty dramatic. I think this happens because noise pushes values in arbitrary directions, while rank swapping at least keeps values close to where they originally were in terms of rank. Noise might make more sense if you need formal privacy guarantees (it connects to differential privacy) but for a practical use case like this, the utility cost is hard to justify when rank swapping gives almost the same risk reduction.

If I had to pick one method for this dataset, I'd go with rank swapping at p=10%. The risk reduction is solid, the accuracy is still usable, and it doesn't mess with the distributions. Not perfect, but the best tradeoff I found.